Ps1:

Step 1 — Confirm Python is installed

```
python --version
```

Step 2 — Create a project folder

```
mkdir C:\qiskit
```

```
cd C:\qiskit
```

Step 3 — Create and activate the virtual environment

```
python -m venv qiskit_env
```

```
qiskit_env\Scripts\Activate.ps1
```

Step 4 — Install the required packages

```
pip install --trusted-host pypi.org --trusted-host files.pythonhosted.org --trusted-host pypi.python.org qiskit qiskit-aer jupyter matplotlib pylatexenc
```

Step 5 — Register the environment as a Jupyter kernel

```
python -m ipykernel install --user --name=qiskit_env --display-name "Python (qiskit_env)" --sys-prefix
```

Step 6 — Launch Jupyter and select the kernel

```
jupyter notebook
```

code:

```
from qiskit import QuantumCircuit

from qiskit.quantum_info import Statevector

qc = QuantumCircuit(1)

qc.h(0)

print(qc)

state = Statevector.from_instruction(qc)

print(state)
```

ps2: H → S → H → t

```
!pip install qiskit

!pip install qiskit_aer

from qiskit import *

from qiskit.quantum_info import *

from qiskit_aer import AerSimulator

from qiskit.visualization import plot_histogram


qc = QuantumCircuit(1)

qc.h(0)

qc.s(0)

qc.h(0)

qc.t(0)

print(qc)

state = Statevector.from_instruction(qc)

print("Final Statevector:", state)

qc.measure_all()

backend = AerSimulator()

result = backend.run(transpile(qc, backend), shots=1024).result()

counts = result.get_counts()

print(counts)

plot_histogram(counts)
```

ps3:

si+:

```
!pip install qiskit qiskit-aer pylatexenc
from qiskit import *
from qiskit.quantum_info import *
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

print(qc)

state = Statevector.from_instruction(qc)
print("Statevector:", state)
qc.measure_all()
backend = AerSimulator()
result = backend.run(transpile(qc, backend), shots=1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

ps4: si-:

```
!pip install qiskit qiskit-aer pylatexenc
from qiskit import *
```

```
from qiskit.quantum_info import *
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
qc = QuantumCircuit(2)

qc.h(0)
qc.cx(0, 1)
qc.x(1)
qc.z(0)

print(qc)

state = Statevector.from_instruction(qc)
print("Statevector:", state)
qc.measure_all()
backend = AerSimulator()
result = backend.run(transpile(qc, backend), shots=1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

ps5:fi+:

```
from qiskit import *
from qiskit.quantum_info import *
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

qc = QuantumCircuit(2)
```

```
qc.h(0)
qc.cx(0, 1)

state = Statevector.from_instruction(qc)
print("Statevector:", state)

qc.measure_all()

backend = AerSimulator()
result = backend.run(transpile(qc, backend), shots=1024).result()
counts = result.get_counts()

print(counts)
plot_histogram(counts)
```

ps6: fi-:

```
!pip install qiskit qiskit-aer pylatexenc
from qiskit import *
from qiskit.quantum_info import *
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
qc = QuantumCircuit(2)

qc.h(0)
qc.cx(0, 1)
qc.z(0)
```

```
print(qc)


state = Statevector.from_instruction(qc)
print("Statevector:", state)
qc.measure_all()
backend = AerSimulator()
result = backend.run(transpile(qc, backend), shots=1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

ps 9: swap gate between 1st-3rd qubit:

for 1st and 3rd qubit swap :

```
qc.cx(0, 2)
qc.cx(2, 0)
qc.cx(0, 2)
```

and for input 010 ➔ qc.x(1)

for 011-→ qc.x(1) , qc.x(2)

ps 10: swap gate between 1st-2nd qubit:

```
  !pip install qiskit qiskit-aer -q
!pip install pylatexenc


from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
```

```
# Create a 3-qubit circuit
qc = QuantumCircuit(3, 3)

# Input state |101>
qc.x(0)
qc.x(2)

# SWAP qubit 0 and qubit 1 using 3 CNOT gates
qc.cx(0, 1)
qc.cx(1, 0)
qc.cx(0, 1)

# Measure all qubits
qc.measure([0, 1, 2], [0, 1, 2])

# Display circuit
print(qc.draw())

# Run circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

# Display histogram
counts = result.get_counts()
display(plot_histogram(counts))
print(counts)
```

ps 11,12 ,13: superdence:

for 00 :

```
!pip install qiskit
!pip install qiskit_aer
from qiskit import *
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

qc = QuantumCircuit(2, 2)

# Create Bell state
qc.h(0)
qc.cx(0, 1)

# Alice encodes message 00
# No gate required

# Bob decodes
qc.cx(0, 1)
qc.h(0)
```

```
# Measure
qc.measure([0, 1], [0, 1])

print(qc)

backend = AerSimulator()
result = backend.run(transpile(qc, backend), shots=1024).result()
counts = result.get_counts()

print(counts)
plot_histogram(counts)
```

for 01: qc.z(0)

for 10: qc.x(0)

for 11 : qc.z(0) qc.x(0)

ps14:

Deutsch–Jozsa Algorithm- balanced function using three qubits

```
!pip install qiskit qiskit-aer -q

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

# Create a 3-qubit circuit
```

```
qc = QuantumCircuit(3, 2)

# Prepare ancilla qubit
qc.x(2)

# Apply Hadamard gates
qc.h(0)
qc.h(1)
qc.h(2)

# Balanced oracle
qc.cx(0, 2)

# Apply Hadamard gates to input qubits
qc.h(0)
qc.h(1)

# Measure input qubits
qc.measure(0, 0)
qc.measure(1, 1)

# Display circuit
print(qc.draw())

# Execute circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
```

```
# Display measurement outcomes
counts = result.get_counts()
print(counts)

# Plot histogram
display(plot_histogram(counts))
```

ps 15: constant dj

```
!pip install qiskit qiskit-aer -q

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

# Create circuit with 3 qubits and 2 classical bits
qc = QuantumCircuit(3, 2)

# Prepare ancilla qubit
qc.x(2)

# Apply Hadamard gates
qc.h(0)
qc.h(1)
qc.h(2)

# Constant oracle (no CNOT gates)
```

```
# Apply Hadamard gates again to input qubits
qc.h(0)
qc.h(1)

# Measure input qubits
qc.measure(0, 0)
qc.measure(1, 1)

# Display circuit
print(qc.draw())

# Run circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

# Display results
counts = result.get_counts()
print(counts)

# Plot histogram
display(plot_histogram(counts))
```

ps 16: balanced on 4 qubit :

```
!pip install qiskit qiskit-aer -q

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

```
from IPython.display import display

# Create circuit: 4 qubits, 3 classical bits
qc = QuantumCircuit(4, 3)

# Prepare ancilla qubit
qc.x(3)

# Apply Hadamard gates
qc.h(0)
qc.h(1)
qc.h(2)
qc.h(3)

# Balanced oracle
qc.cx(0, 3)

# Apply Hadamard gates again to input qubits
qc.h(0)
qc.h(1)
qc.h(2)

# Measure input qubits
qc.measure(0, 0)
qc.measure(1, 1)
qc.measure(2, 2)

# Display circuit
```

```
print(qc.draw())

# Run circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

# Display results
counts = result.get_counts()
print(counts)

# Plot histogram
display(plot_histogram(counts))
```

ps17 : dj constant on 4 qubit

```
!pip install qiskit qiskit-aer -q

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

# Create circuit with 4 qubits and 3 classical bits
qc = QuantumCircuit(4, 3)

# Prepare ancilla qubit
qc.x(3)

# Apply Hadamard gates
```

```
qc.h(0)
qc.h(1)
qc.h(2)
qc.h(3)

# Constant oracle: no CNOT gates

# Apply Hadamard gates again to input qubits
qc.h(0)
qc.h(1)
qc.h(2)

# Measure input qubits
qc.measure(0, 0)
qc.measure(1, 1)
qc.measure(2, 2)

# Display circuit
print(qc.draw())

# Run circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

# Display results
counts = result.get_counts()
print(counts)
```

Plot histogram

display(plot_histogram(counts))

ps 18 : df for constant and balanced for 2 qubit :

constant:

```
!pip install qiskit qiskit-aer -q
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

# Create circuit
qc = QuantumCircuit(2, 1)

# Prepare ancilla
qc.x(1)

# Apply Hadamard gates
qc.h(0)
qc.h(1)

# Constant oracle: no CNOT gate

# Apply Hadamard to input qubit
qc.h(0)

# Measure
qc.measure(0, 0)
```

```
# Display circuit
print(qc.draw())

# Run circuit
result = AerSimulator().run(qc, shots=1024).result()
counts = result.get_counts()

print(counts)
display(plot_histogram(counts))
```

balanced:

```
!pip install qiskit qiskit-aer -q

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

# Create circuit
qc = QuantumCircuit(2, 1)

# Prepare ancilla
qc.x(1)

# Apply Hadamard gates
qc.h(0)
qc.h(1)
```

```
# Balanced oracle
qc.cx(0, 1)

# Apply Hadamard to input qubit
qc.h(0)

# Measure
qc.measure(0, 0)

# Display circuit
print(qc.draw())

# Run circuit
result = AerSimulator().run(qc, shots=1024).result()
counts = result.get_counts()

print(counts)
display(plot_histogram(counts))
```

ps19: grovers for t=|11> two qubit :

```
!pip install qiskit qiskit-aer -q

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
```

```
# Create a 2-qubit circuit
qc = QuantumCircuit(2, 2)

# Step 1: Apply Hadamard gates
qc.h(0)
qc.h(1)

# Step 2: Oracle to mark state |11>
qc.cz(0, 1)

# Step 3: Apply Hadamard gates
qc.h(0)
qc.h(1)

# Step 4: Apply X gates
qc.x(0)
qc.x(1)

# Step 5: Apply CZ gate
qc.cz(0, 1)

# Step 6: Apply X gates again
qc.x(0)
qc.x(1)

# Step 7: Apply Hadamard gates again
qc.h(0)
```

```
qc.h(1)

# Step 8: Measure both qubits
qc.measure([0, 1], [0, 1])

# Display circuit
print(qc.draw())

# Run circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

# Display output
counts = result.get_counts()
print(counts)

# Plot histogram
display(plot_histogram(counts))
```

for t=00 change qc.cz(0,1) to :

```
qc.x(0)
qc.x(1)
qc.cz(0, 1)
qc.x(0)
qc.x(1)
```

for t=01 change qc.cz(0,1) to :

```
qc.x(1); qc.cz(0, 1); qc.x(1)
```

for t=10 change qc.cz(0,1) to :

```
qc.x(0); qc.cz(0, 1); qc.x(0)
```

ps20: shors:

```
!pip install qiskit qiskit-aer qiskit-algorithms -q


from qiskit_aer import AerSimulator
from math import gcd


# Simple classical demonstration of Shor's period-finding method
def shor_factor(N):
    print("\nNumber:", N)


    if N < 2:
        print("Enter a valid composite number")
        return


    # Handle even numbers
    if N % 2 == 0:
        print("Factors:", 2, "and", N // 2)
        return


    # Try values of a
    for a in range(2, N):
        g = gcd(a, N)


        if g > 1:
            print("Factors:", g, "and", N // g)
            return
```

```
        # Find period r such that a^r mod N = 1
        x = 1
        for r in range(1, N + 1):
            x = (x * a) % N

            if x == 1:
                break

        # Period must be even
        if r % 2 == 0:
            factor1 = gcd(a**(r // 2) - 1, N)
            factor2 = gcd(a**(r // 2) + 1, N)

            if factor1 not in (1, N) and factor2 not in (1, N):
                print("Period:", r)
                print("Factors:", factor1, "and", factor2)
                return

    print("Try another value of a")



# Test the given numbers
for N in [15, 39, 72]:
    shor_factor(N)

for different no change in :
for N in [15, 39, 72]:
    shor_factor(N)
```